

GREEDY AND DYNAMIC PROGRAMMING

8. Sherlock and Cost

```
#include <stdio.h>

#include <stdlib.h>

int main()
{
    int T, N, B, L, R, ML, MR, X, Y, P, Q;

    scanf("%d", &T);
    for(int i = 0; i < T; i++) {
        scanf("%d", &N);
        for(int j = 0; j < N; j++) {
            scanf("%d", &B);
            if(j) {
                X = L - 1 + ML;
                Y = R - 1 + MR;
                P = abs(L - B) + ML;
                Q = abs(R - B) + MR;
                ML = (X > Y ? X : Y);
                MR = (P > Q ? P : Q);
            }
            else {
                ML = MR = 0;
            }
            L = 1;
            R = B;
        }
        printf("%d\n", (ML > MR ? ML : MR));
    }
    return 0;
}
```

```
}
```

9. Marc's Cakewalk

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <math.h>
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp;
```

```
    temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
int partition(int *x, int start, int end)
```

```
{
```

```
    int pivot, pindex, i;
```

```
    pivot = x[end];
```

```
    pindex = start;
```

```
    for(i = start; i < end; i++) {
```

```
        if(x[i] >= pivot) {
```

```
            swap(&x[i], &x[pindex]);
```

```
            pindex++;
```

```
        }
```

```
    }
```

```
    swap(&x[pindex], &x[end]);
```

```
    return pindex;
```

```
}
```

```
void quicksort(int *x, int start, int end) {
```

```

    if(start < end) {
        int i = partition(x, start, end);
        quicksort(x, start, i - 1);
        quicksort(x, i + 1, end);
    }
}

int main() {
    int n, *calories;
    int i;
    long long sum = 0;
    scanf("%d", &n);
    calories = malloc(sizeof(int) * n);
    for(i = 0; i < n; i++) {
        scanf("%d", &calories[i]);
    }
    quicksort(calories, 0, n - 1);
    for(i = 0; i < n; i++) {
        sum += (long long)calories[i] * ((long long)pow(2, i));
    }
    printf("%lld", sum);
    return 0;
}

```

RANGE QUERIES

13. Prefix Sum Array

```
#include <stdio.h>
```

```

void display(int arr[], int n) {
    int i;

```

```

    for(i = 0; i < n; i++) {
        printf("\t%d", arr[i]);
    }
}

void create_prefix_sum_array(int arr[], int n) {
    int i;
    for(i = 1; i < n; i++) {
        arr[i] = arr[i] + arr[i - 1];
    }
}

void sum(int arr[], int a, int b) {
    int result;
    if(a == 0)
        result = arr[b];
    else
        result = arr[b] - arr[a - 1];
    printf("The sum is %d", result);
}

int main() {
    int n, i, arr[10], a, b;
    printf("Enter the number of elements\n");
    scanf("%d", &n)
    printf("Enter the array elements\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("\nOriginal array");
    display(arr, n);
    create_prefix_sum_array(arr, n);
}

```

```

printf("\nPrefix sum array");
display(arr, n);
printf("\nEnter the range to find the sum\n");
scanf("%d%d", &a, &b);
sum(arr, a, b);
return 0;
}

```

14. Fenwick Tree Construction

```

#include <stdio.h>

int sum(int arr[], int a, int b) {
    int i, sum = 0;
    for(i = a; i <= b; i++) {
        sum += arr[i];
    }
    return sum;
}

void create_fenwick_tree(int T[], int arr[], int n) {
    int a, b, k;
    for(k = 1; k <= n; k++) {
        a = k - (k & -k) + 1;
        b = k;
        T[k] = sum(arr, a, b);
    }
}

void display(int arr[], int n) {
    int i;
    for(i = 1; i <= n; i++) {
        printf("\t%d", arr[i]);
    }
}

```

```

    }
}

int main() {
    int n, i, arr[100], T[100];
    printf("Enter the number of elements\n");
    scanf("%d", &n);
    printf("Enter the array elements\n");
    for(i = 1; i <= n; i++) {
        scanf("%d", &arr[i]);
    }
    printf("\nOriginal array");
    display(arr, n);
    create_fenwick_tree(T, arr, n);
    printf("\nFenwick Tree");
    display(T, n);
    return 0;
}

```

15. Fenwick Tree - Sum of Elements

```

#include <stdio.h>

void sum(int T[], int k) {
    int s = 0;
    while(k >= 1) {
        s += T[k];
        k -= k & -k;
    }
    printf("\nThe sum is %d", s);
}

void display(int arr[], int n) {

```

```

int i;
for(i = 1; i <= n; i++) {
    printf("\t%d", arr[i]);
}
}

int main() {
    int n = 8, k;
    int T[100] = {0, 1, 4, 4, 16, 6, 7, 4, 29};
    printf("\nFenwick Tree");
    display(T, n);
    printf("\nEnter the value for k\n");
    scanf("%d", &k);
    sum(T, k);
    return 0;
}

```

ADDITIONAL PROGRAMS

RECURSION AND BIT MANIPULATION

3. Maximizing XOR

```

#include <stdio.h>

int maxXor(int l, int r) {
    int max = 0, i, j;
    for(i = l; i < r; i++) {
        for(j = i + 1; j <= r; j++)
            max = max<(i^j)?i^j:max;
    return max;
}

int main() {
    int l, r;

```

```
scanf("%d", &l);  
scanf("%d", &r);  
printf("%d", maxXor(l, r));  
return 0;  
}
```

4. Sum vs XOR

```
#include <stdio.h>  
  
int main() {  
    long long int n, m = 1;  
    scanf("%lld", &n);  
    while(n > 0) {  
        if(n % 2 == 0)  
            m *= 2;  
        n /= 2;  
    }  
    printf("%lld\n", m);  
    return 0;  
}
```

5. Flipping Bits

```
#include <stdio.h>  
  
int main() {  
    int t;  
    unsigned int n;  
    scanf("%d", &t);  
    while(t-- > 0) {  
        scanf("%u", &n);  
        printf("%u\n", ~n);  
    }  
}
```



```
}  
  
return 0;  
  
}
```

GREEDY AND DYNAMIC PROGRAMMING

6. Mark and Toys

```
#include <stdio.h>  
  
void quicksort(int x[100000], int first, int last) {  
    int pivot, j, temp, i;  
    if(first < last) {  
        pivot = first;  
        i = first;  
        j = last;  
        while(i < j) {  
            while(x[i] <= x[pivot] && i < last)  
                i++;  
            while(x[j] > x[pivot])  
                j--;  
            if(i < j) {  
                temp = x[i];  
                x[i] = x[j];  
                x[j] = temp;  
            }  
        }  
        temp = x[pivot];  
        x[pivot] = x[j];  
        x[j] = temp;  
        quicksort(x, first, j - 1);  
        quicksort(x, j + 1, last);  
    }
```

```

    }
}

int main() {
    int n, k, i, avail = 0, count = 0;

    scanf("%d", &n);
    scanf("%d", &k);
    int cost[n];
    for(i = 0; i < n; i++) {
        scanf("%d", &cost[i]);
    }
    quicksort(cost, 0, n - 1);
    while(avail <= k) {
        avail += cost[count];
        count++;
    }
    printf("%d\n", count - 1);
    return 0;
}

```

7. Greedy Florist

```

#include <stdio.h>
#include <stdlib.h>

int a[200], i, k, sum, n;

int com(const void *xx, const void *yy) {
    if(*(int *)xx > *(int *)yy)
        return -1;
    return 1;
}

int main() {

```

```

scanf("%d %d", &n, &k);
for(i = 0; i < n; i++) {
    scanf("%d", &a[i]);
}
qsort(a, n, sizeof(a[0]), com);
sum = 0;
for(i = 0; i < n; i++) {
    sum += a[i] * (1 + i / k);
}
printf("%d\n", sum);
return 0;
}

```

STRINGS

8. Camelcase

```

#include <stdio.h>
#include <string.h>
int main() {
    char s[1000];
    int count = 1, i;
    scanf("%s", s);
    for(i = 0; i < strlen(s); i++) {
        if(s[i] >= 'A' && s[i] <= 'Z') {
            count++;
        }
    }
    printf("%d\n", count);
    return 0;
}

```

9. Mars Exploration

```
#include <stdio.h>

int main() {
    char S[1000];
    int i, count = 0;
    scanf("%s", S);
    for(i = 0; S[i] != '\0'; i += 3) {
        if(S[i] != 'S')
            count++;
        if(S[i + 1] != 'O')
            count++;
        if(S[i + 2] != 'S')
            count++;
    }
    printf("%d", count);
    return 0;
}
```

10. Funny String

```
#include <stdio.h>
#include <string.h>
#include <math.h>

int main() {
    int T, N, funny, i, r;
    char S[10001];
    scanf("%d", &T);
    while(T--) {
        scanf("%s", S);
```

```

N = strlen(S);

funny = 1;

for(i = 1, r = N - 2; i < N; i++, r--) {
    if(fabs(S[i] - S[i - 1]) != fabs(S[r] - S[r + 1])) {
        funny = 0;
        break;
    }
}

if(funny)
    printf("Funny\n");
else
    printf("Not Funny\n");
}

return 0;
}

```

11. Alternating Characters

```

#include <stdio.h>

int main() {
    int t;

    int i, count = 0;

    char c[100002];

    scanf("%d", &t);

    while(t--) {
        scanf("%s", c);

        for(i = 0; c[i] != '\0'; i++) {
            if(c[i] == c[i + 1]) {
                count++;
            }
        }
    }
}

```

```
    }  
    printf("%d\n", count);  
    count = 0;  
}  
return 0;  
}
```

12. Beautiful Binary String

```
#include <stdio.h>  
  
int main() {  
    int n;  
    char B[1000];  
    int i = 0, count = 0;  
    scanf("%d", &n);  
    scanf("%s", B);  
    while(B[i]) {  
        if(B[i] == '0' && B[i + 1] == '1' && B[i + 2] == '0') {  
            B[i + 2] = '1';  
            count++;  
        }  
        i++;  
    }  
    printf("%d", count);  
    return 0;  
}
```

RANGE QUERIES

13. Fenwick Tree – Add a value

```
#include <stdio.h>

void display(int arr[], int n) {
    int i;
    for(i = 1; i <= n; i++) {
        printf("\t%d", arr[i]);
    }
}

void add(int T[], int n, int k, int x) {
    while(k <= n) {
        T[k] += x;
        k += k & -k;
    }
    display(T, n);
}

int main() {
    int n = 8, k, x;
    int T[100] = {0, 1, 4, 4, 16, 6, 7, 4, 29};
    printf("\nFenwick Tree");
    display(T, n);
    printf("\nEnter the position and value to be increased\n");
    scanf("%d%d", &k, &x);
    add(T, n, k, x);
    return 0;
}
```